Activaciones Racionales Condicionadas por Contexto: ¿Cuándo Conviene Adaptar la No Linealidad de una Red?

Agustín H. Astie Forconesi Juan P. Gerino Joaquín I. Ortega Miguel Méndez Garabetti

Laboratorio de Investigación en Ciencia y Tecnología,
Universidad del Aconcagua, Mendoza, Argentina.

agustinastie3@gmail.com, juanpigerino@gmail.com, joaquinimanolortega@gmail.com, mmendez@uda.edu.ar

Resumen

Las redes neuronales profundas aplican la misma función de activación a todas sus entradas, sin importar de qué tipo de datos provengan. Cuando un único modelo debe atender regímenes heterogéneos —imágenes de distintas fuentes, texto de varios dominios, señales capturadas en condiciones diferentes— esa transformación única puede volverse un cuello de botella. Este trabajo investiga si conviene generar la forma de la no linealidad a partir del contexto de cada muestra, y a qué costo. Se propone la Activación Racional Condicionada por Contexto (CCA): una red auxiliar pequeña produce, para cada ejemplo, los coeficientes de una curva racional estable que en la inicialización coincide de manera exacta con la activación SiLU, de modo que el modelo nunca parte de una situación peor que la de referencia. La propuesta se compara con cinco alternativas, entre ellas Dynamic ReLU-B y Meta-ACON, igualando parámetros y operaciones, sobre diez semillas emparejadas y con pruebas exactas corregidas por comparaciones múltiples. En un banco real formado por MNIST, SVHN y USPS, sin informar nunca al modelo de qué conjunto proviene cada imagen, CCA alcanza 88,68 % *de exactitud y supera a la mejor alternativa por* 0,62 *puntos porcentuales. Una matriz de doce tareas adicionales muestra dónde ocurre eso: la ventaja se concentra en las que exigen transformaciones distintas por régimen y se desvanece en las homogéneas. El sobrecosto no es aritmético sino de lanzamiento de núcleos, y la compilación recupera la mayor parte. El resultado es una regla de uso concreta antes que una mejora universal.*

Palabras clave—Aprendizaje profundo, Funciones de activación adaptativas, Cómputo condicionado, Hiperredes, Evaluación con presupuesto igualado, Cambio de dominio.

1. Introducción

El aprendizaje profundo debe buena parte de su éxito a unos pocos componentes simples repetidos millones de veces. Entre ellos, la función de activación —la no linealidad que se aplica elemento por elemento entre dos capas— es el más elemental: sin ella, una red de muchas capas colapsa en una única transformación lineal. Desde la adopción masiva de la unidad rectificadora [1], la práctica dominante consiste en elegir una curva fija —ReLU, GELU [2] o SiLU [3]— y aplicarla de forma idéntica a todas las muestras, capas y situaciones. Esa uniformidad es cómoda, barata y, para datos homogéneos, suficiente.

Los sistemas que se despliegan fuera del laboratorio, en cambio, rara vez observan un único tipo de datos. Un lector de importes recibe cifras manuscritas de un formulario y también fotografías de carteles tomadas con un teléfono; un modelo acústico procesa grabaciones limpias y otras saturadas de ruido. Cada una de esas condiciones —a las que llamaremos *regímenes*— puede requerir una transformación diferente de las mismas características internas, y una activación fija obliga a resolverlas todas con una única curva, es decir, con un compromiso. El problema, además, no se reduce a reconocer el régimen: aun identificándolo, la red necesita *modificar el cómputo* que aplica en consecuencia.

La literatura ofrece dos respuestas parciales. La primera es aprender la forma de la activación durante el entrenamiento, como hacen PReLU [4], Maxout [5], Swish [6], las unidades racionales [7, 8] o las redes de Kolmogórov–Arnold [9]; en todos esos casos la curva se aprende una sola vez y queda congelada en inferencia, de modo que sigue siendo la misma para todos los regímenes. La segunda proviene del cómputo condicionado [10], donde una parte de la red genera o selecciona parámetros de otra: hiperredes [11], modulación FiLM [12], núcleos convolucionales dependientes de la muestra [13] o mezclas de expertos [14]. En la intersección se ubican Dynamic ReLU [15] y Meta-ACON [16], que condicionan la activación misma al con-

tenido de la entrada.

Ese cruce deja un vacío que motiva este trabajo. Las ventajas publicadas se miden casi siempre sobre grandes arquitecturas de visión, entre métodos que no comparten presupuesto de parámetros ni de operaciones, y casi nunca vienen acompañadas de controles negativos que muestren dónde la técnica *no* sirve. Uno termina sabiendo que el método ganó, pero no por qué ni a cambio de qué. Falta una respuesta empírica a una pregunta práctica: *¿bajo qué condiciones una no linealidad condicionada por el contexto supera a una fija y a las alternativas dinámicas, cuando todas disponen del mismo presupuesto y el régimen debe inferirse de los datos, sin metadatos que lo revelen?*

Para responderla proponemos la *Activación Racional Condicionada por Contexto* (CCA, por sus siglas en inglés), en la que una red auxiliar diminuta genera, para cada muestra, los coeficientes de una curva racional estable. La utilidad esperada es concreta: mejorar la exactitud de un modelo desplegado sobre datos heterogéneos sin agrandarlo, sin etiquetas de dominio y sin entrenar un modelo por régimen. El aporte se evalúa con dos niveles de evidencia separados: una comparación confirmatoria con presupuesto igualado sobre semillas jamás usadas durante el diseño, y una matriz de doce tareas de visión, lenguaje, audio, razonamiento y generación que incluye controles diseñados para que la técnica pueda fallar.

Las contribuciones del trabajo son cuatro:

- una activación condicionada por contexto que se inicializa *exactamente* como SiLU y no puede, por lo tanto, partir en desventaja;
- una comparación contra Dynamic ReLU-B, MetaACON y una racional estática con parámetros y operaciones igualados, diez semillas emparejadas y pruebas exactas corregidas;
- evidencia en una aplicación real multi-dominio (MNIST, SVHN y USPS) donde el contexto se infiere de la propia red;
- una delimitación explícita del alcance: en qué tareas la técnica no aporta nada, y cuánto cuesta en tiempo de cómputo.

El artículo se organiza así: la sección *Elementos del Trabajo y Metodología* presenta la activación propuesta, los métodos con los que se la compara y el protocolo experimental; la sección *Resultados* responde, en ese orden, si la propuesta supera a las alternativas bajo presupuesto igualado, en qué tareas lo hace, de dónde proviene la ganancia y cuánto cuesta; la sección *Trabajos Relacionados* discute críticamente los antecedentes, y la última resume los aportes, las limitaciones y las líneas de continuación.

2. Elementos del Trabajo y Metodología

2.1. La activación propuesta

Sea x el valor de una preactivación, es decir, la salida de una capa lineal o convolucional antes de aplicar la no linealidad, y sea $c \in \mathbb{R}^m$ un vector de *contexto* que resume la situación de la muestra en proceso: en las tareas de imagen es el promedio espacial de las preactivaciones del bloque y en las de texto el promedio de los vectores de palabra de la ventana observada. En ningún caso c contiene la etiqueta ni la identidad del dominio; se calcula sólo con información que el modelo ya posee.

Un generador G —un perceptrón de una capa oculta, con unos pocos cientos de parámetros— transforma ese contexto en los coeficientes de la curva; para limitarlo, los canales se reparten en g grupos y los de un mismo grupo comparten coeficientes. La activación se calcula en dos pasos. Primero se aplica una transformación afín condicionada,

$$u = w(c)\,x + d(c), \tag{1}$$

donde $w(c)$ es un factor de escala y $d(c)$ un desplazamiento, ambos producidos por G; la variable u es la preactivación ya reencuadrada según el contexto. Sobre ella se aplica la curva propiamente dicha, que denotamos A_{L} por corresponder a la variante CCA-Lite:

$$A_{\mathrm{L}}(x,c) = u\,\sigma(u) + \alpha\,\frac{P(u,c)}{1+Q(u,c)^2}, \tag{2}$$

$$\begin{aligned} P(u,c) &= a_0(c) + a_1(c)\,u + a_2(c)\,u^2, \\ Q(u,c) &= b_0(c) + b_1(c)\,u. \end{aligned} \tag{3}$$

El primer término, $u\,\sigma(u)$ con σ la función logística, es exactamente SiLU [3] evaluada en u y actúa como base estable. El segundo es una corrección racional cuyo numerador P y denominador Q son polinomios de grado bajo en u: los cinco coeficientes $a_0(c)$, $a_1(c)$, $a_2(c)$, $b_0(c)$ y $b_1(c)$ los produce G a partir del contexto, mientras que α es una amplitud aprendida por grupo que no depende de la muestra, inicializada en 1. Como el denominador nunca vale menos que uno, la fracción no tiene polos y la activación es derivable en toda la recta, aunque no acotada. La transformación (1) es una modulación al estilo FiLM [12], y el contexto por promedio espacial coincide con el descriptor de Squeeze-and-Excitation [17]. *CCA-Lite* usa $g = 16$ grupos y una capa oculta de 16 unidades.

Una curva racional de grado bajo aproxima con dificultad los cambios bruscos de pendiente. Por eso la variante *CCA-Hinge*, que notamos A_{H}, agrega un tercer término:

$$A_{\mathrm{H}}(x,c) = A_{\mathrm{L}}(x,c) + \lambda(c)\,h(u,c), \tag{4}$$

donde $h(u,c) = \sqrt{(u-t(c))^2+\varepsilon}$ y $\varepsilon = 10^{-4}$ es una constante pequeña que suaviza el vértice y lo hace derivable. La amplitud $\lambda(c)$ y el umbral $t(c)$ también provienen del generador, de modo que el término describe una bisagra cuyo

punto de quiebre se desplaza según el contexto; equivale a la componente de valor absoluto con la que puede reescribirse el máximo de dos ramas lineales de Maxout [5].

Una propiedad de diseño articula todo lo anterior: la última proyección de G se inicializa en cero, de modo que al comenzar el entrenamiento $w = 1$, $d = 0$, y tanto los cinco coeficientes racionales como la amplitud λ valen cero. Reemplazando esos valores en (1)–(4) se obtiene $A(x,c) = x\sigma(x)$: la red arranca siendo *exactamente* la red con SiLU. CCA es entonces un superconjunto estricto del modelo de referencia, no puede partir en desventaja, y cualquier diferencia posterior es atribuible a lo aprendido.

Esta inicialización tiene una consecuencia: con $Q = 0$, el gradiente $-2PQ/(1+Q^2)^2$ respecto de Q es nulo, y b_0 y b_1 permanecen exactamente en cero durante todo el entrenamiento, lo que verificamos. En la práctica la corrección es entonces cuadrática, $\alpha\,(a_0 + a_1 u + a_2 u^2)$; la Rational estática se comporta igual. Iniciar b en un valor pequeño no nulo activaría el denominador sin perder la equivalencia con SiLU. La configuración de CCA-Hinge (16 grupos, generador de ancho 16) se eligió con las semillas de desarrollo 2001–2003 y validación separada, y se congeló antes de la confirmación: ningún resultado de prueba intervino en esa elección.

2.2. Métodos de comparación

Todos los métodos comparten la misma arquitectura anfitriona, la misma interfaz de activación y, cuando son adaptativos, exactamente el mismo vector de contexto c. Las diferencias quedan así confinadas a la no linealidad.

SiLU [3] es la activación fija de referencia, $x\sigma(x)$, y **SiLU igualada** es la misma curva sobre una red algo más ancha, que consume los parámetros gastados por los métodos adaptativos en su generador; sin ella, cualquier ventaja podría atribuirse a un modelo más grande. **Rational** tiene la forma de (2) pero con coeficientes aprendidos una sola vez y compartidos por todas las muestras [7, 8]; aísla el efecto de la *racionalidad* del de la *condicionalidad*. **Dynamic ReLU-B** [15] genera desde el contexto las pendientes e interceptos de dos ramas lineales y devuelve su máximo, compartiendo los parámetros en el eje espacial y adaptándolos por canal; es el comparador más fuerte de la literatura reciente. **Meta-ACON** [16] interpreta la activación como una transición suave entre dos ramas lineales y predice desde el contexto cuán abrupta es esa transición. Finalmente, **CCA-Lite** y **CCA-Hinge** son las variantes propuestas.

A ellos se suma un control que no es una alternativa competitiva sino un instrumento de diagnóstico, al que llamamos **inyección directa**: usa el mismo codificador de contexto que CCA, pero en lugar de deformar la curva suma el vector c a la representación de entrada. Recibe la misma información; lo que cambia es *dónde* se la aplica, de modo que permite separar el aporte del mecanismo del aporte de la información.

2.3. Protocolo de comparación justa

La credibilidad de una comparación entre activaciones depende de que las diferencias medidas no provengan del presupuesto ni del azar; adoptamos tres resguardos.

El primero es el presupuesto igualado. Se toma la configuración de CCA-Hinge como objetivo de recursos y una búsqueda discreta ajusta el ancho de la red anfitriona, la cantidad de grupos y el tamaño del generador de cada competidor hasta quedar dentro del 105 % de sus parámetros y del 110 % de sus operaciones por muestra. Esa búsqueda observa únicamente el costo, nunca la exactitud de prueba.

El segundo es el emparejamiento de semillas: todos los métodos se entrenan con las mismas diez semillas, de modo que cada comparación es pareada y la variabilidad entre inicializaciones —una fuente de error bien documentada [18]— afecta a ambos lados por igual. No se aplica detención temprana ni ajuste de hiperparámetros por método, y el optimizador es AdamW con idéntica configuración para todos.

El tercero es el tratamiento estadístico. Informamos media, desviación estándar e intervalo de confianza del 95 % por remuestreo (10 000 réplicas). Las diferencias de CCA-Hinge contra cada competidor se evalúan con una prueba de permutación pareada exacta bilateral, se corrigen por familia con el procedimiento de Holm [19] y se acompañan del tamaño de efecto d_z de Cohen. Conviene detenerse en la granularidad de esos valores, porque explica varias cifras que se repiten más adelante. Una prueba exacta sobre diez pares sólo puede devolver múltiplos de $2/2^{10} = 0{,}0020$, de modo que 0,0020 es el menor p bruto posible, y Holm lo multiplica por el tamaño de la familia. Cada familia agrupa las comparaciones de CCA-Hinge dentro de una misma tarea: son cinco —SiLU igualada, Rational, Dynamic ReLU-B, Meta-ACON y CCA-Lite— en las tareas donde no corre el control de inyección directa, y seis donde sí corre, esto es, en lenguaje por caracteres, en las tres de generación de imágenes y en razonamiento modular. El piso corregido queda así en 0,0098 con cinco comparaciones y en 0,0117 con seis. Cuando alguno de esos dos valores aparece en los resultados hay que leerlo como el mínimo que el diseño permite alcanzar y no como una coincidencia; cualquier otro valor informado es un p genuino.

2.4. Tareas, datos y contexto

El banco de pruebas combina datos reales y sintéticos. Los reales son MNIST [20], Fashion-MNIST [21], CIFAR-10 [22], SVHN [23], USPS [24], el conjunto de dígitos hablados FSDD [25] y las reseñas multi-dominio de Amazon [26]; los sintéticos son una tarea de texto cifrado y otra de razonamiento modular, que permiten fijar con exactitud cuántos regímenes existen. Las redes anfitrionas son deliberadamente pequeñas: una CNN de dos bloques para imagen y audio, un perceptrón de dos capas sobre bolsa de vectores para texto, un transformador de tres capas para el lenguaje por caracteres y una convolución causal local para

la generación de imágenes. Todas se entrenan con AdamW y tasa 2×10^{-3} (3×10^{-3} en las dos generativas), con particiones fijadas antes de entrenar. El contexto c se obtiene, según la tarea, como promedio espacial de la preactivación de cada bloque, promedio de los vectores de la ventana observada o histograma causal de los niveles ya generados; las identidades de dominio se conservan sólo para informar métricas por dominio y nunca alimentan al modelo. El detalle de particiones, pasos y tamaños de lote acompaña al código publicado.

Dos tareas cumplen un papel especial: la aplicación real multi-dominio, que mezcla MNIST, SVHN y USPS en un único conjunto de diez clases, donde el modelo debe clasificar dígitos sin saber si la imagen proviene de un formulario, de una fotografía callejera o de un envío postal; y los *controles de régimen único*, versiones de las tareas sintéticas con un solo régimen, donde la hipótesis predice que la adaptación no debería aportar nada.

3. Resultados

3.1. ¿Supera a las alternativas con el mismo presupuesto?

La Tabla 1 presenta la comparación confirmatoria sobre las semillas 3001–3010, que no se usaron en ninguna etapa de diseño. Todos los métodos quedan a menos del 1 % del presupuesto de parámetros —excepto CCA-Lite, que queda por debajo ($-1{,}32$ % en texto y $-4{,}12$ % en dígitos)— y dentro del 6 % en operaciones, de modo que ninguna diferencia puede atribuirse a que un modelo sea más grande que otro. El ajuste, en cambio, no fue simétrico: CCA-Hinge se configuró con las semillas de desarrollo y los competidores usaron su configuración por defecto, lo que favorece a la propuesta.

En la tarea de texto cifrado, CCA-Hinge y CCA-Lite alcanzan 94,75 % y Dynamic ReLU-B 94,87 %. La diferencia pareada es de $-0{,}12$ puntos porcentuales, con intervalo $[-0{,}36;\ 0{,}10]$, $d_z = -0{,}31$ y $p_{\text{Holm}} = 0{,}773$: las tres son estadísticamente indistinguibles. La comparación que sí resulta significativa es la que enfrenta a CCA-Hinge con la SiLU igualada, a la que supera por 4,17 puntos ($[3{,}38;\ 4{,}98]$, $d_z = 3{,}08$, $p_{\text{Holm}} = 0{,}0098$, el piso de esta familia de cinco comparaciones). La lectura es doble: condicionar la activación aporta mucho frente a una curva fija, pero en esta tarea no aporta nada frente a otras formas de condicionarla.

El resultado cambia en la aplicación real. Sobre la mezcla de MNIST, SVHN y USPS, CCA-Hinge alcanza 88,68 % y supera a Dynamic ReLU-B por 0,62 puntos ($[0{,}21;\ 1{,}11]$, $d_z = 0{,}81$, $p_{\text{Holm}} = 0{,}0234$), a MetaACON por 5,10 y a la racional estática por 5,54 (ambas con $p_{\text{Holm}} = 0{,}0098$). También supera a CCA-Lite por 0,57 ($[0{,}30;\ 0{,}86]$, $p_{\text{Holm}} = 0{,}0117$). El 0,0234 no es un piso sino un p genuino: es el valor exacto sin corregir de esa comparación, la menos extrema de su familia, a la que Holm asigna multiplicador 1. El desglose por dominio explica de dónde sale la ventaja: obtiene 96,71 % en MNIST, 75,01 % en SVHN y 94,34 % en USPS, frente a 96,34 %, 73,52 % y 94,32 % de Dynamic ReLU-B. Casi toda la mejora se concentra en SVHN, el régimen visualmente más distinto —fotografías con fondo, iluminación variable y dígitos vecinos—, mientras que en los dos conjuntos de trazo manuscrito el empate es técnico. Para un sistema desplegado esto significa que la técnica actúa donde el modelo compartido sufre más, y que lo hace sin recibir jamás la identidad del conjunto de origen: la infiere de sus propias activaciones.

3.2. ¿En qué tareas ayuda adaptar la curva?

La Tabla 2 extiende la comparación a doce escenarios que conservan la misma red anfitriona, y la Figura 1 la resume desde la pregunta que interesa: cuánto gana CCA-Hinge frente al mejor competidor, tarea por tarea, y si esa diferencia resiste el análisis estadístico.

Las cinco tareas de clasificación marcadas con ‡ se recorrieron con el mismo procedimiento de presupuesto igualado de la sección anterior, sobre las semillas 4001–4010; en las cinco la familia de Holm es de cinco comparaciones. Las restantes no están igualadas; la columna ΔP_{DyR} indica la diferencia de parámetros. Dos filas repiten tareas ya presentes en la Tabla 1 con otro bloque de semillas: cuando importe una cifra puntual, la que corresponde citar es la confirmatoria, porque proviene de semillas que no intervinieron en ninguna etapa de diseño.

El patrón es nítido y tiene una explicación única. Las ganancias grandes aparecen donde los regímenes exigen transformaciones genuinamente distintas: 20,11 puntos en razonamiento con ocho claves de sustitución ($p_{\text{Holm}} = 0{,}0117$, familia de seis) y 1,44 en la mezcla real de dígitos ($p_{\text{Holm}} = 0{,}0098$, familia de cinco), ambas significativas (en dígitos, con presupuesto igualado, la ventaja es de 0,62). En razonamiento la varianza es alta —CCA-Hinge va de 38,4 % a 68,5 % según la semilla—, aunque su peor semilla supera a ocho de las diez de Dynamic ReLU-B. Las tareas multirrégimen restantes no sostienen ninguna diferencia una vez igualado el presupuesto: en audio la ventaja de 1,27 puntos sobre Dynamic ReLU-B es direccional pero no alcanza el umbral ($p_{\text{Holm}} = 1{,}000$) y la mejor media pertenece a la racional estática; en imágenes multirrégimen hay empate ($-0{,}02$, $p_{\text{Holm}} = 0{,}957$) y en texto multirrégimen la diferencia es negativa y, esta vez, significativa: Dynamic ReLU-B aventaja a CCA-Hinge por 0,25 puntos ($[-0{,}37;\ -0{,}13]$, $d_z = -1{,}25$, $p_{\text{Holm}} = 0{,}0312$, familia de cinco). La magnitud es pequeña —un cuarto de punto sobre una exactitud del 96 %— pero el signo importa y conviene declararlo: en la tarea de texto multirrégimen la envolvente lineal por tramos basta, y la corrección racional no compra nada. La comparación confirmatoria de la Tabla 1 sobre texto cifrado, con semillas ajenas al diseño, deja a ambos métodos indistinguibles ($-0{,}12$, $p_{\text{Holm}} = 0{,}773$), de modo

Tabla 1. Comparación confirmatoria sobre las semillas 3001–3010 y costo de esas mismas configuraciones de presupuesto igualado. Exactitud media ± desviación estándar, con intervalo de confianza del 95 %; ΔP y ΔF: diferencias porcentuales de parámetros y de operaciones respecto de CCA-Hinge; t: milisegundos por iteración de propagación y gradiente (lote 256, GPU NVIDIA L4) en modo interpretado y compilado; S: aceleración obtenida al compilar.

Tarea	Método	Exactitud [IC95 %]	ΔP	ΔF	t eager	t comp.	S
Texto cifrado	SiLU igualada	90,58±3,33 [88,59; 92,49]	+0,73	−5,94	1,251	1,427	0,88
	Rational	94,51±2,36 [93,08; 95,85]	+0,81	−1,10	3,576	2,048	1,75
	Dynamic ReLU-B	**94,87±1,97** [93,68; 95,98]	+0,59	−4,65	3,308	2,326	1,42
	Meta-ACON	94,53±2,30 [93,12; 95,80]	+0,17	−4,73	3,302	2,519	1,31
	CCA-Lite	94,75±2,01 [93,55; 95,89]	−1,32	−2,89	5,584	2,614	2,14
	CCA-Hinge	94,75±2,13 [93,50; 95,98]	0	0	5,943	2,637	**2,25**
MNIST + SVHN + USPS	SiLU igualada	81,14±1,17 [80,41; 81,81]	−0,66	+1,63	1,569	1,706	0,92
	Rational	83,14±0,79 [82,68; 83,60]	−0,34	+2,96	6,256	2,507	2,50
	Dynamic ReLU-B	88,06±0,98 [87,47; 88,63]	+0,93	+0,15	5,173	2,871	1,80
	Meta-ACON	83,58±1,03 [83,01; 84,20]	−0,08	+0,02	4,381	2,976	1,47
	CCA-Lite	88,11±0,68 [87,74; 88,54]	−4,12	−4,39	7,164	3,017	2,37
	CCA-Hinge	**88,68±0,50** [88,38; 88,98]	0	0	9,065	3,057	**2,97**

que la lectura prudente es que en texto CCA-Hinge no supera a Dynamic ReLU-B. Una parte de las ventajas que la matriz mostraba antes de igualar recursos era, por lo tanto, atribuible al tamaño.

El caso más informativo es el que no funciona. En el análisis de sentimiento sobre cuatro dominios reales de Amazon, la mejor media pertenece a la SiLU fija (82,79 %) y ninguna diferencia de CCA-Hinge resulta significativa. No es un fracaso del experimento sino una confirmación de la hipótesis: esos dominios difieren en vocabulario, no en la transformación que hay que aplicar sobre las características internas. Cuando el régimen cambia *qué* palabras aparecen pero no *cómo* procesarlas, adaptar la curva no aporta nada. Los controles de régimen único apuntan en la misma dirección, y con presupuesto igualado el resultado es limpio. Con un solo régimen los seis métodos quedan entre 99,43 % y 99,65 % en texto y entre 98,31 % y 98,50 % en imágenes, y ninguna diferencia resiste la corrección: frente a Dynamic ReLU-B, CCA-Hinge queda en $-0{,}01$ puntos en imágenes ($p_{\text{Holm}} = 0{,}906$). Los mismos dos métodos se separan por 0,62 puntos cuando hay tres regímenes reales. Al desaparecer los regímenes desaparece lo que la adaptación aporta; los datos no sostienen que agregar parámetros dinámicos mejore por sí mismo.

En las cuatro tareas generativas la ventaja reaparece con tamaños de efecto grandes. CCA-Hinge obtiene el menor número de bits por dimensión en MNIST, Fashion-MNIST y CIFAR-10, mejorando a Dynamic ReLU-B en 0,011 ([0,010; 0,013], $d_z = 4{,}02$), 0,043 ([0,039; 0,047], $d_z = 6{,}11$) y 0,026 ([0,024; 0,028], $d_z = 6{,}32$), las tres con $p_{\text{Holm}} = 0{,}0117$. En lenguaje por caracteres, en cambio, la reducción de perplejidad de 11,985 a 11,723 (0,261; [0,043; 0,483], $d_z = 0{,}71$) no resiste la corrección de Holm ($p_{\text{Holm}} = 0{,}113$) y ambos métodos quedan indistinguibles. En las cuatro, además, CCA-Lite iguala a CCA-Hinge ($p_{\text{Holm}} \geq 0{,}146$): la bisagra, decisiva en dígitos, no aporta ventaja medible en generación y basta la variante más barata. Vale la pena separar aquí significancia de relevancia. Los tamaños de efecto son grandes porque la varianza entre semillas es muy baja, no porque la diferencia lo sea: las tres ganancias representan entre el 1,2 % y el 3,2 % de la métrica del competidor, equivalentes a entre uno y diez bytes por imagen codificada. Son mejoras reales y reproducibles, pero de una magnitud que difícilmente se perciba en un sistema desplegado. Es la misma lectura que propone este trabajo en general: la evidencia estadística indica dónde el mecanismo actúa, no que su efecto sea grande en todos los casos en que actúa.

3.3. ¿De dónde proviene la ganancia?

Que un método gane no explica por qué gana, y con una activación condicionada hay al menos dos explicaciones alternativas plausibles: que la ganancia venga de la información de contexto y no del mecanismo, o que el generador ni siquiera se aparte de su inicialización. Tres experimentos permiten descartarlas y atribuir la mejora al mecanismo propuesto.

El primero responde si alcanza con *disponer* del contexto. El control de inyección directa recibe el mismo vector c del mismo codificador, pero lo suma a la entrada en lugar de usarlo para deformar la curva, y rinde marcadamente peor: 13,88 de perplejidad frente a 11,72 en lenguaje por caracteres, 12,28 % frente a 49,88 % en razonamiento, y 0,955, 1,350 y 1,470 bits por dimensión en generación frente a 0,937, 1,284 y 1,430. Las cuatro diferencias son significativas sobre diez semillas ($p_{\text{Holm}} = 0{,}0117$). La información, por sí sola, no produce la ganancia: importa dónde se la aplica. Este control no separa, sin embargo, el reencuadre afín (1) de la deformación de la curva; esa ablación queda pendiente.

El segundo verifica que el modelo use el contexto *correcto*. Al barajar los vectores de contexto entre muestras del mismo lote, CCA-Hinge cae de 88,05 % a 57,61 % en imágenes multirrégimen y de 88,96 % a 81,50 % en la

Tabla 2. Matriz completa sobre diez semillas emparejadas (media ± desviación estándar): semillas 4001–4010 en las filas de clasificación y de lenguaje por caracteres, y 4101–4110 en las tres de generación de imágenes. Las filas marcadas con ‡ se recorrieron bajo el mismo protocolo de presupuesto igualado de la Tabla 1, de modo que su columna SiLU corresponde a la SiLU igualada; en ellas ningún competidor excede los parámetros de CCA-Hinge en más del $1{,}3\,\%$, y sólo CCA-Lite se aparta por debajo de manera apreciable (hasta $-4{,}12\,\%$). En las tareas generativas la métrica es menor-mejor; en el resto, exactitud en porcentaje. Rat. = racional estática, DyR = Dynamic ReLU-B, Meta = Meta-ACON. ΔP_{DyR}: parámetros de Dynamic ReLU-B respecto de CCA-Hinge (%).

Tarea (métrica)	SiLU	Rat.	DyR	Meta	CCA-Lite	CCA-Hinge	ΔP_{DyR}
Lenguaje por caracteres (PPL ↓)	21,703±,318	21,644±,496	11,985±,211	12,885±,551	11,763±,197	**11,723±,360**	−0,69
Generación MNIST (bits/dim ↓)	1,083±,007	1,069±,008	,949±,006	,951±,006	,938±,006	**,937±,007**	−3,24
Generación Fashion (bits/dim ↓)	1,597±,009	1,567±,008	1,327±,009	1,335±,007	1,286±,010	**1,284±,009**	−3,24
Generación CIFAR-10 (bits/dim ↓)	1,588±,008	1,581±,007	1,456±,005	1,469±,007	1,430±,005	**1,430±,007**	−2,64
Imagen, 8 regímenes‡	85,56±,51	87,24±,32	**88,07±,62**	87,00±,47	87,90±,71	88,05±,71	+0,93
Imagen, 1 régimen (control)‡	98,31±,09	98,41±,21	**98,50±,17**	98,41±,12	98,45±,24	98,49±,23	+0,93
Texto, 8 regímenes‡	93,42±1,54	96,28±1,16	**96,67±,98**	96,34±1,14	96,37±,99	96,42±1,03	+0,59
Texto, 1 régimen (control)‡	99,43±,34	99,56±,19	**99,65±,16**	99,53±,20	99,59±,16	99,55±,22	+0,59
Dígitos reales, 3 dominios†	81,25±1,12	83,42±,98	87,52±,89	83,22±1,74	88,54±,55	**88,96±,72**	−11,04
Razonamiento, 8 claves	8,21±,51	16,09±,52	29,77±8,21	23,24±3,56	47,10±10,08	**49,88±9,45**	−3,33
Audio, 6 regímenes‡	73,92±2,40	**75,13±2,42**	73,07±2,37	73,82±2,50	74,27±1,74	74,33±1,55	−0,21
Sentimiento, 4 dominios reales	**82,79±,58**	82,43±,49	82,31±,84	82,30±,48	82,04±,70	82,48±,48	−1,39

† Misma tarea que en la Tabla 1, con otro bloque de semillas. ‡ Presupuesto igualado, semillas 4001–4010.

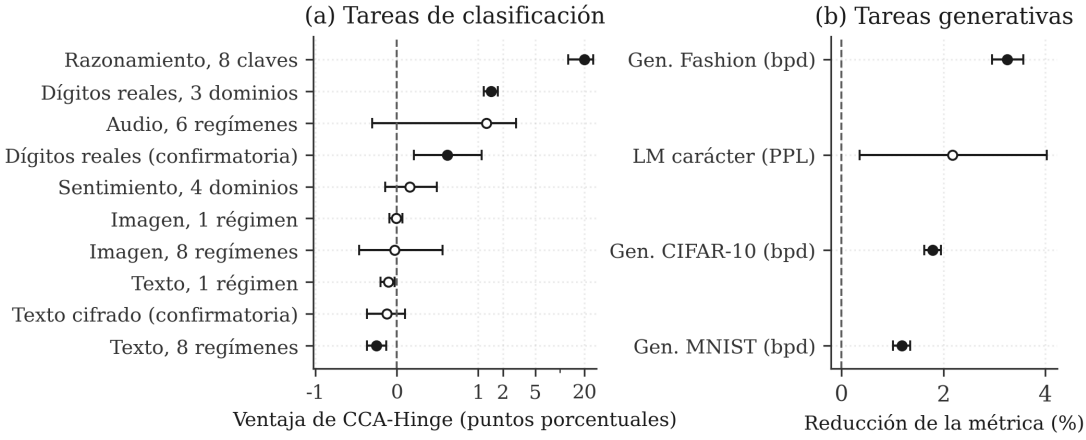


Figura 1. Ventaja de CCA-Hinge sobre Dynamic ReLU-B en cada tarea, con intervalos de confianza del $95\,\%$ sobre diez semillas emparejadas. Las entradas rotuladas *confirmatoria* provienen de la Tabla 1 (semillas 3001–3010) y el resto de la Tabla 2. El marcador relleno indica diferencia significativa tras la corrección de Holm y el hueco, no significativa; el eje de (a) está comprimido de manera logarítmica más allá de ± 1. Las ganancias se concentran donde los regímenes exigen transformaciones distintas; en las tareas homogéneas el intervalo cruza el cero.

mezcla real de dígitos, mientras que en el control de régimen único la misma perturbación casi no produce efecto ($98{,}49\,\%$ a $98{,}46\,\%$), que es lo que cabe esperar si no hay regímenes que distinguir. La activación depende, entonces, del resumen que corresponde a cada muestra; la prueba no demuestra que el modelo identifique el régimen en un sentido semántico, sino que su cómputo está efectivamente condicionado.

El tercero observa el interior del mecanismo. La Figura 2 muestra, para un modelo CCA-Hinge entrenado en texto cifrado, las curvas generadas ante cinco regímenes, su diferencia respecto de la curva promedio y la varianza de cada coeficiente. Se observan tres hechos: la curva aprendida se aleja notablemente de SiLU, de modo que el generador no permanece en su inicialización; las curvas de distintos regímenes difieren entre sí, sobre todo donde la preactivación es grande en valor absoluto; y la adaptación se concentra en los coeficientes del numerador —a_2 es el más variable—. Los del denominador valen $b_0 = b_1 = 0$, por la inicialización descrita en la sección 2.1, no por una preferencia aprendida.

3.4. ¿Cuánto cuesta?

Una activación más compleja sólo resulta útil si su costo es tolerable, y conviene ser explícito al respecto porque es la objeción más inmediata a la propuesta. Las tres últimas columnas de la Tabla 1 miden el tiempo de una iteración de propagación y gradiente con lotes de 256 muestras, sobre esas mismas configuraciones de presupuesto igualado, en una GPU NVIDIA L4, en modo interpretado (*eager*) y compilado con `torch.compile` [27].

El sobrecosto es de lanzamiento, no aritmético. En

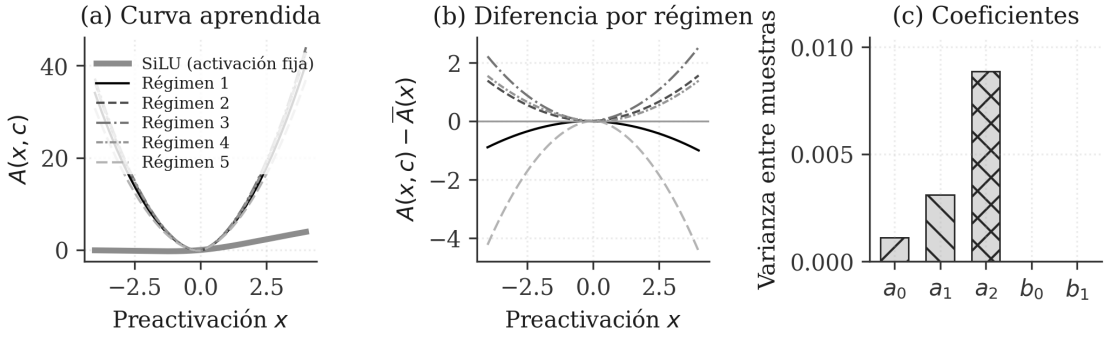


Figura 2. Mecanismo interno de CCA-Hinge en la tarea de texto cifrado. (a) Curva generada para cinco regímenes frente a la SiLU fija. (b) Diferencia de cada curva respecto del promedio, que hace visible la adaptación. (c) Varianza de cada coeficiente entre muestras: la red modifica el numerador; el denominador queda en $b_0 = b_1 = 0$.

modo interpretado, CCA-Hinge cuesta 4,75 veces lo que la SiLU igualada en texto y 5,78 veces en dígitos. Esa penalización no proviene del trabajo aritmético: sus operaciones por muestra son apenas un 6,3 % mayores en texto y un 1,6 % *menores* en dígitos, donde la búsqueda de presupuesto compensa el generador con una red anfitriona más angosta. La distancia entre ambos factores —casi seis veces más lento con la misma cantidad de cuentas— mide el costo de ejecutar decenas de operaciones puntuales diminutas como núcleos de GPU separados.

La compilación recupera la mayor parte. Al fusionar esas operaciones, CCA-Hinge obtiene la mayor aceleración de la tabla (2,25× y 2,97×), por ser el método con más trabajo puntual fusionable; el efecto crece con la complejidad de la activación, y la SiLU igualada, que no tiene nada que fusionar, incluso se degrada (0,88× y 0,92×). El sobrecosto compilado frente a ella baja así a 1,85× y 1,79×, y frente a Dynamic ReLU-B queda en +13,4 % y +6,5 %: sobre la mezcla real de dígitos, ese 6,5 % adicional compra 0,62 puntos de exactitud frente al mejor competidor y 7,5 frente a la SiLU igualada.

Alcance de la medición. La compilación reduce el costo en estado estacionario, no el de estas corridas: su calentamiento de 20 a 27 segundos no se amortiza en entrenamientos cortos. La exactitud no depende del hardware y se conserva de la corrida original; el costo sí, y por eso se remidió en una sola máquina. No afirmamos que estas proporciones se mantengan en otras GPU ni en modelos mayores.

4. Trabajos Relacionados

La idea de que la no linealidad no tiene por qué ser fija no es nueva, y ha seguido dos caminos que hasta ahora corrieron bastante separados y que este trabajo articula.

El primero busca *aprender* la curva, y la escala del problema es considerable: un relevamiento reciente cataloga más de cuatrocientas funciones de activación propuestas en tres décadas [28], lo que da una medida tanto del interés sostenido en el tema como de la dificultad de establecer cuáles aportan una mejora real. PReLU [4] aprende la pendiente de la rama negativa y muestra que un grado mínimo de libertad ya mejora el resultado; Maxout [5] generaliza la idea tomando el máximo de varias ramas lineales, a costa de multiplicar los parámetros; y la búsqueda automática de activaciones [6] derivó en Swish y en la adopción de curvas suaves como SiLU [3] o GELU [2]. Las unidades racionales aportan un argumento teórico —aproximan funciones suaves con menor profundidad que una red ReLU equivalente [7]— las de Padé muestran que pueden entrenarse de extremo a extremo [8] y su uso en aprendizaje por refuerzo sugiere que la plasticidad de la curva importa sobre todo cuando la distribución de entrada cambia [29], mientras que las redes de Kolmogórov–Arnold [9] aprenden una función univariada en cada arista, aunque evaluaciones posteriores que controlan parámetros y costo encuentran que su ventaja se limita a la regresión simbólica y no se sostiene en visión ni en lenguaje [30], un antecedente que ilustra por qué aquí insistimos en el presupuesto igualado. Todas comparten una limitación: la curva es única para todo el conjunto y queda congelada en inferencia, de modo que un modelo entrenado sobre regímenes heterogéneos debe resolverlos con un único compromiso.

El segundo camino es el cómputo condicionado, donde una parte de la red produce o selecciona parámetros de otra en función de la entrada [10]; el área está sistematizada en el relevamiento de redes neuronales dinámicas [31], que distingue el condicionamiento por muestra, por región espacial y por profundidad, y dentro del cual CCA pertenece al primero. Las hiperredes [11] generan los pesos de la red principal; FiLM [12] modula las características intermedias mediante escalas y desplazamientos condicionados; CondConv [13] combina núcleos convolucionales con pesos que dependen de la muestra; y las mezclas de expertos dispersas [14] encaminan cada ejemplo hacia un subconjunto de especialistas, un esquema hoy vigente en modelos de lenguaje de gran escala [32]. Todas condicionan los pesos o las representaciones, pero dejan intacta la forma de la no linealidad, que es el componente sobre el que actúa CCA.

La distinción no es sólo conceptual: el control de inyección directa recibe la misma información de contexto y la incorpora a la representación —como haría una modulación al estilo FiLM—, y aun así rinde significativamente peor, lo que muestra que el punto de aplicación importa.

En la intersección de ambos caminos se ubican los comparadores directos de este trabajo. Dynamic ReLU [15] genera desde un resumen global los coeficientes de dos ramas lineales y devuelve su máximo, con lo que obtiene una envolvente lineal por tramos adaptada a la entrada; ACON y su versión Meta-ACON [16] reinterpretan la activación como una transición suave entre dos ramas y condicionan el factor que la regula. CCA se diferencia en que condiciona a la vez un reencuadre afín y una corrección racional sin polos, y en que CCA-Hinge incorpora una base de quiebre suave que recupera la capacidad de Maxout sin renunciar a la inicialización exacta en SiLU. Nuestra comparación implementa las tres formulaciones sobre una interfaz común y les entrega idéntico contexto, condición que rara vez se cumple en los reportes originales.

Queda un aspecto metodológico que ninguna de estas líneas resuelve: las ventajas de las activaciones adaptativas suelen informarse sin igualar el presupuesto y sin análisis estadístico pareado, prácticas cuya influencia sobre las conclusiones está documentada [18]. Parte de la contribución de este artículo es mostrar qué queda de esas ventajas bajo presupuesto igualado, semillas emparejadas, corrección por comparaciones múltiples [19] y controles negativos explícitos; el hallazgo de que la técnica no mejora a una activación fija en el problema real de sentimiento es tan parte del resultado como las ganancias multirrégimen. CCA tampoco compite con las técnicas de adaptación de dominio o de transferencia, que requieren conocer o etiquetar los dominios: aquí el régimen se infiere de la propia actividad de la red.

5. Conclusión y Trabajos Futuros

Este trabajo se propuso determinar si conviene adaptar la no linealidad de una red al contexto de cada muestra, y a qué costo. La respuesta es afirmativa, pero acotada, y esa delimitación —más que la ganancia en sí— es su principal aporte. Bajo presupuesto igualado y sobre semillas no utilizadas en el diseño, CCA-Hinge supera significativamente a Dynamic ReLU-B en el banco real formado por MNIST, SVHN y USPS. Sin igualar el presupuesto también lo supera en las tres tareas de generación de imágenes, por márgenes pequeños, y en razonamiento modular. En audio, con presupuesto igualado, queda por detrás de la racional estática. En texto queda empatada con los métodos adaptativos en la tarea confirmatoria y un cuarto de punto por debajo de Dynamic ReLU-B en la matriz con presupuesto igualado, y en el análisis de sentimiento multi-dominio no mejora a una SiLU fija. La regla que emerge es orientativa: condicionar la forma de la activación conviene cuando el contexto modifica la transformación necesaria, y agregar complejidad dinámica a un problema homogéneo no garantiza beneficio alguno. Esta regla se formuló a posteriori, y la sola presencia de varios regímenes no la predice (imagen y audio multirrégimen no mejoran); definir criterios medibles antes de entrenar queda pendiente. Los controles indican que la mejora proviene del condicionamiento y no del mero acceso a la información.

El estudio tiene limitaciones que conviene explicitar. Las redes anfitrionas son pequeñas, de decenas de miles de parámetros, y no evaluamos modelos fundacionales, por lo que las proporciones de costo y de ganancia no pueden extrapolarse sin más. La generación de imágenes se limita a 500 pasos, un régimen corto que privilegia la rapidez del experimento. La equivalencia estricta de recursos se verifica en las dos tareas de la Tabla 1 y en las cinco de clasificación marcadas en la Tabla 2; las restantes conservan la red anfitriona original, con las diferencias de parámetros que informa esa tabla. Además, sólo CCA-Hinge se ajustó en validación, y el denominador racional no se entrenó. Tampoco evaluamos cambios de dominio no observados durante el entrenamiento, un escenario frecuente en producción.

Cada limitación sugiere una continuación: activar el denominador con una inicialización no nula; aislar el reencuadre afín; ajustar a los competidores en validación; replicar la comparación con presupuesto igualado sobre transformadores de tamaño medio, donde el costo relativo del generador es menor; medir el comportamiento ante regímenes nunca vistos, que es donde la inferencia del contexto podría fallar; escribir un núcleo fusionado a medida, ya que la penalización es de lanzamiento y no aritmética; y caracterizar qué codifica el vector de contexto, pues el barajado demuestra dependencia del contexto correcto pero no identificación semántica del régimen.

El código, las configuraciones, los resultados por semilla, los manifiestos SHA-256 y los comandos de reproducción están disponibles en el repositorio público del proyecto.[1] Las corridas de exactitud usaron PyTorch 2.11 sobre una NVIDIA RTX 4070 Laptop; las diez semillas de generación y las mediciones de costo, PyTorch 2.8 sobre una NVIDIA L4, y cada bloque de diez semillas proviene de una única máquina. Los datos no se redistribuyen: se descargan de sus fuentes originales conservando las particiones descritas.

Referencias

[1] V. Nair and G. E. Hinton, "Rectified linear units improve restricted boltzmann machines," in *Proceedings of the 27th International Conference on Machine Learning*, pp. 807–814, 2010.

[2] D. Hendrycks and K. Gimpel, "Gaussian error linear units

[1] `https://github.com/agustinastie2/Context-Conditioned-Rational-Activations`

(GELUs)," *arXiv preprint arXiv:1606.08415*, 2016.

[3] S. Elfwing, E. Uchibe, and K. Doya, "Sigmoid-weighted linear units for neural network function approximation in reinforcement learning," *Neural Networks*, vol. 107, pp. 3–11, 2018.

[4] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification," in *Proceedings of the IEEE International Conference on Computer Vision*, pp. 1026–1034, 2015.

[5] I. J. Goodfellow, D. Warde-Farley, M. Mirza, A. Courville, and Y. Bengio, "Maxout networks," in *Proceedings of the 30th International Conference on Machine Learning*, pp. 1319–1327, 2013.

[6] P. Ramachandran, B. Zoph, and Q. V. Le, "Searching for activation functions," *arXiv preprint arXiv:1710.05941*, 2017.

[7] N. Boullé, Y. Nakatsukasa, and A. Townsend, "Rational neural networks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020.

[8] A. Molina, P. Schramowski, and K. Kersting, "Padé activation units: End-to-end learning of flexible activation functions in deep networks," in *International Conference on Learning Representations*, 2020.

[9] Z. Liu, Y. Wang, S. Vaidya, *et al.*, "KAN: Kolmogorov–Arnold networks," *arXiv preprint arXiv:2404.19756*, 2024.

[10] Y. Bengio, N. Léonard, and A. Courville, "Estimating or propagating gradients through stochastic neurons for conditional computation," *arXiv preprint arXiv:1308.3432*, 2013.

[11] D. Ha, A. Dai, and Q. V. Le, "Hypernetworks," in *International Conference on Learning Representations*, 2017.

[12] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville, "FiLM: Visual reasoning with a general conditioning layer," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018.

[13] B. Yang, G. Bender, Q. V. Le, and J. Ngiam, "CondConv: Conditionally parameterized convolutions for efficient inference," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[14] N. Shazeer, A. Mirhoseini, K. Maziarz, *et al.*, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *International Conference on Learning Representations*, 2017.

[15] Y. Chen, X. Dai, M. Liu, D. Chen, L. Yuan, and Z. Liu, "Dynamic relu," in *Computer Vision – ECCV 2020*, pp. 351–367, 2020.

[16] N. Ma, X. Zhang, M. Liu, and J. Sun, "Activate or not: Learning customized activation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 8032–8042, 2021.

[17] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 7132–7141, 2018.

[18] X. Bouthillier, P. Delaunay, M. Bronzi, *et al.*, "Accounting for variance in machine learning benchmarks," in *Proceedings of Machine Learning and Systems*, vol. 3, pp. 747–769, 2021.

[19] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.

[20] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[21] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms," *arXiv preprint arXiv:1708.07747*, 2017.

[22] A. Krizhevsky, "Learning multiple layers of features from tiny images," tech. rep., University of Toronto, 2009.

[23] Y. Netzer, T. Wang, A. Coates, *et al.*, "Reading digits in natural images with unsupervised feature learning," in *NIPS Workshop on Deep Learning and Unsupervised Feature Learning*, 2011.

[24] J. J. Hull, "A database for handwritten text recognition research," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 16, no. 5, pp. 550–554, 1994.

[25] Z. Jackson, C. Souza, J. Flaks, Y. Pan, H. Nicolas, and A. Thite, "Free spoken digit dataset," 2018.

[26] J. Blitzer, M. Dredze, and F. Pereira, "Biographies, bollywood, boom-boxes and blenders: Domain adaptation for sentiment classification," in *Proceedings of the 45th Annual Meeting of the Association of Computational Linguistics*, pp. 440–447, Association for Computational Linguistics, 2007.

[27] J. Ansel, E. Yang, H. He, *et al.*, "PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation," in *Proc. ASPLOS*, pp. 929–947, 2024.

[28] V. Kunc and J. Kléma, "Three decades of activations: A comprehensive survey of 400 activation functions for neural networks," *arXiv preprint arXiv:2402.09092*, 2024.

[29] Q. Delfosse, P. Schramowski, M. Mundt, A. Molina, and K. Kersting, "Adaptive rational activations to boost deep reinforcement learning," in *International Conference on Learning Representations*, 2024.

[30] Y. Hou, T. Ji, D. Zhang, and A. Stefanidis, "Kolmogorov–Arnold networks: A critical assessment of claims, performance, and practical viability," *arXiv preprint arXiv:2407.11075*, 2024.

[31] Y. Han, G. Huang, S. Song, L. Yang, H. Wang, and Y. Wang, "Dynamic neural networks: A survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 11, pp. 7436–7456, 2022.

[32] A. Q. Jiang, A. Sablayrolles, A. Roux, *et al.*, "Mixtral of experts," *arXiv preprint arXiv:2401.04088*, 2024.

Datos de Contacto

Agustín H. Astie Forconesi. Universidad del Aconcagua, Catamarca 147, M5500 Mendoza, Argentina.
`agustinastie3@gmail.com`